Lecture Notes: Multi-Layer Perceptrons

CIC1205 - Machine Learning

Eduardo Bezerra

May 3, 2026

0 Contents

Before You Read **3**

Learning Objectives **3**

1 Introduction **4**
1.1 From Linear Models to Neural Networks . . . 4
1.2 Motivation: The XOR Problem . . . 4

2 Architecture **6**
2.1 The Biological Neuron Analogy . . . 6
2.2 Layers and the MLP Architecture . . . 7
2.3 Capacity and Depth . . . 8

3 Activation Functions **9**
3.1 The Role of Non-Linearity . . . 9
3.2 Common Activation Functions . . . 9

4 Forward Pass **11**
4.1 Layer-by-Layer Computation . . . 12
4.2 Worked Example: Forward Pass in a 2–2–1 Network . . . 12

5 Loss Functions **13**
5.1 Measuring Prediction Error . . . 13
5.2 Common Loss Functions . . . 13

6 Training: Gradient Descent and Backpropagation **15**
6.1 The Training Objective . . . 15
6.2 Gradient Descent . . . 16
6.3 Backpropagation: Computing Gradients Efficiently . . . 16
6.4 The Chain Rule and Error Signals . . . 17
6.5 The Training Loop: Forward, Backward, and Update . . . 17
6.6 Worked Example: Backward Pass . . . 19

7 Optimization in Practice **21**
7.1 Weight Initialization . . . 21
7.2 Advanced Optimizers . . . 21
7.3 Regularization . . . 22

8 Implementation **24**
8.1 Implementation in Scikit-learn . . . 24
8.2 Implementation in PyTorch . . . 25

9 Common Pitfalls and Debugging **26**
9.1 First Sanity Checks . . . 26
9.2 Checklist for Training MLPs . . . 27
9.3 The Vanishing Gradient Problem . . . 27

10 Connection to Deep Learning **28**

10.1 MLPs as the Foundation of Deep Learning 28
10.2 What Changes at Scale 29

11 Final Remarks **30**
11.1 Summary of Key Concepts 30
11.2 Recommended Next Steps 30

0 Before You Read

These notes assume that you are already familiar with the following topics. If any of them feel unclear, review the corresponding previous notes before studying backpropagation in particular.

- Linear and logistic regression.
- The concept of gradient descent for parameter optimization.
- Train, validation, and test splits; overfitting and underfitting.
- Basic derivatives and the chain rule from calculus.
- Vector and matrix notation, including matrix–vector multiplication.

0 Learning Objectives

After studying these notes, you should be able to:

- explain why hidden layers allow MLPs to model nonlinear decision boundaries;
- compute a forward pass in a small MLP by hand, given weights, biases, and an input vector;
- match output activation functions and loss functions to regression, binary classification, and multi-class classification tasks;
- explain the role of backpropagation in computing gradients efficiently using the chain rule;
- diagnose common training problems such as overfitting, vanishing gradients, and unstable loss curves;
- implement a basic MLP using scikit-learn or PyTorch.

1 Introduction

1.1 From Linear Models to Neural Networks

Classical machine learning methods such as logistic regression and linear support vector machines learn **linear decision boundaries in the feature space provided to them**. They can model nonlinear patterns only if the original inputs are manually transformed into a more suitable representation. For example, through polynomial features or kernel functions. The limitation, therefore, is not that nonlinearity is impossible, but that these models require the practitioner to engineer the right representation explicitly.

The **Multi-Layer Perceptron (MLP)** takes a different approach: it learns useful intermediate representations automatically through its hidden layers. As a result, an MLP can discover nonlinear structure directly from raw input features, without manual feature engineering. This is the key conceptual advance that separates neural networks from classical linear models.

Multi-Layer Perceptron

A **Multi-Layer Perceptron (MLP)** is a feedforward artificial neural network consisting of an input layer, one or more hidden layers, and an output layer. Each layer applies a learned linear transformation followed by a non-linear activation function. The network learns its parameters through a process called **backpropagation**, guided by gradient descent on a loss function.

Historical context. The theoretical foundations of MLPs were established in the 1980s, most notably with the popularization of the backpropagation algorithm by Rumelhart et al. [1986]. For decades, training deep networks remained challenging due to computational constraints and optimization difficulties. The modern *deep learning* era emerged around 2012, when advances in hardware (GPUs), data availability, and architectural innovations (ReLU activations, batch normalization, dropout) made it practical to train networks with tens or hundreds of layers. Understanding MLPs thoroughly is therefore an essential prerequisite for deep learning.

Scope. MLPs can be applied to both classification and regression tasks. They operate on fixed-size vector inputs and produce a vector output. The same fundamental architecture underlies many modern deep learning systems, from convolutional networks (for images) to transformers (for text).

1.2 Motivation: The XOR Problem

A classic example that illustrates the need for hidden layers is the **XOR problem**. The XOR (exclusive-or) function maps two binary inputs to a binary output:

x_1	x_2	$\mathrm{XOR}(x_1, x_2)$
0	0	0
0	1	1
1	0	1
1	1	0

No single straight line can separate the two classes (0 and 1) in this dataset, meaning that a single-layer linear model cannot solve XOR. However, a network with even one hidden layer can solve XOR exactly, because the hidden layer can learn a nonlinear transformation of the input space that makes the classes linearly separable.

Key idea. Hidden layers act as feature detectors: they learn intermediate representations of the data that are progressively more abstract and task-relevant. Crucially, the output layer then solves a simpler problem in this learned representation space, rather than directly in the original input space. Deeper networks learn hierarchical representations, where lower layers detect simple patterns and upper layers combine them into complex concepts.

Common Misconception

A hidden layer does not simply "add more variables" to the model. It learns a new *representation* of the input data. The output layer then solves a simpler problem in this learned representation space. This is why depth matters: each layer can build upon the abstractions of the previous one.

Figure 1 illustrates this key idea geometrically. The hidden layer maps the original input space into a new representation space in which the classes become linearly separable. The output layer then applies a simple linear boundary in this transformed space.

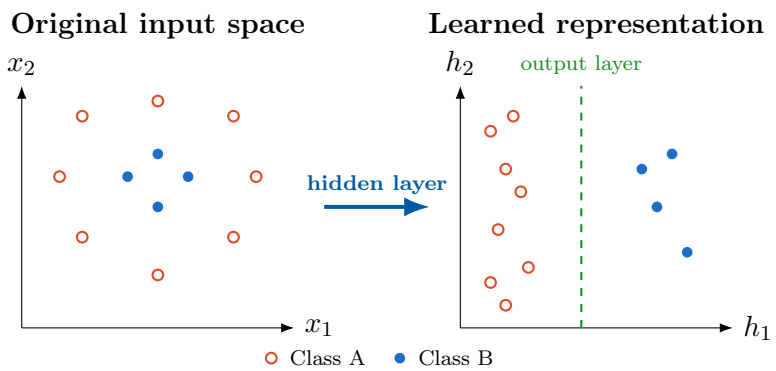

Figure 1: A hidden layer can transform a problem that is not linearly separable in the original input space into one that is linearly separable in a learned representation space. The output layer then applies a simple linear boundary to this transformed representation.

Entry Ticket 1

Consider a binary classification problem where two classes are arranged in concentric circles (inner circle = class 0, outer ring = class 1).

1. Can a logistic regression model solve this problem? Justify your answer geometrically.
2. Informally, how could a hidden layer help? What kind of transformation would allow a subsequent linear layer to separate the classes?
3. Name two other real-world problems where you would expect linear models to fail but neural networks to succeed.

2 Architecture

2.1 The Biological Neuron Analogy

The artificial neuron is loosely inspired by biological neurons in the brain. A biological neuron receives signals from many other neurons through its *dendrites*, combines these signals in the cell body, and, if the combined signal exceeds a threshold, fires an electrical impulse through its *axon* to other neurons. The artificial neuron follows the same pattern: it receives inputs, combines them via a weighted sum, and passes the result through an activation function.

Figure 2 illustrates the main components of a biological neuron and the direction of signal flow. Incoming signals from other neurons are received through the *dendrites* and integrated in the *cell body* or *soma*. If the combined signal is strong enough, the neuron propagates an electrical impulse along the *axon*, which transmits the signal toward the synaptic terminals and, ultimately, to other neurons.

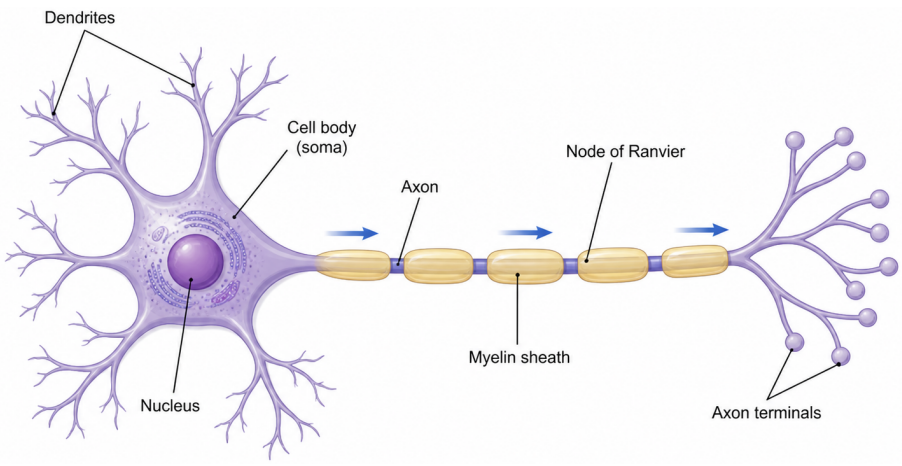

Figure 2: Schematic representation of a biological neuron. The figure was generated by the author using an AI-assisted image generation tool. Dendrites receive incoming signals, the cell body integrates these signals, and the axon transmits the resulting signal toward the axon terminals.

Limits of the Biological Analogy

The biological neuron analogy is useful for building intuition, but it should not be taken literally. Artificial neurons are mathematical functions, not biologically realistic simulations of neurons. The key computational idea in an MLP is not biological realism, but the repeated composition of linear transformations and nonlinear activation functions. Modern deep learning research is almost entirely independent of neuroscience.

Figure 3 shows the basic structure of an artificial neuron. Each input x_i is associated with a weight w_i, and these contributions are combined through the linear expression $z = \sum_{i=1}^{n} w_i x_i + b$, where b is a bias term. The resulting quantity z is then passed through an activation function, producing the final output $y = f(z)$.

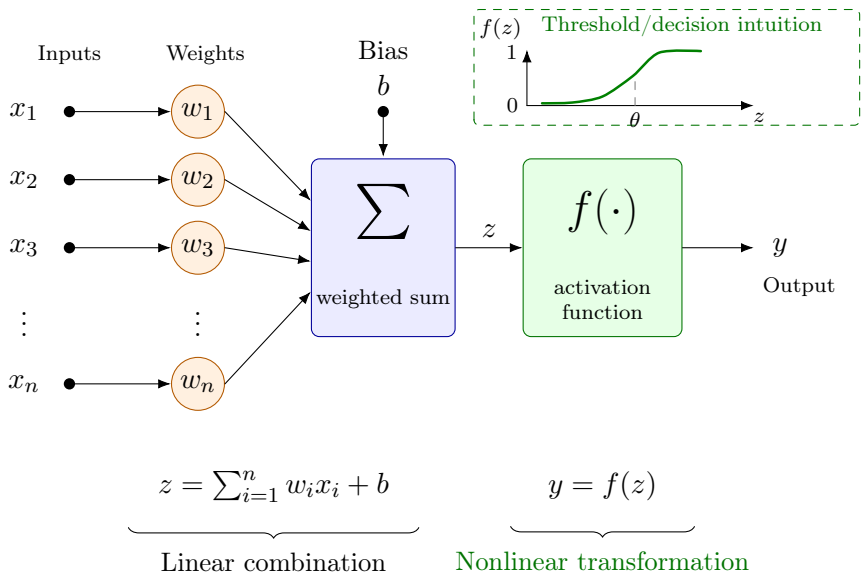

Figure 3: Schematic representation of an artificial neuron. Inputs are multiplied by weights and combined through a weighted sum with bias. The resulting value is then transformed by an activation function to produce the output.

2.2 Layers and the MLP Architecture

An MLP organizes neurons into layers:

- **Input layer**: receives the raw feature vector $\mathbf{x} \in \mathbb{R}^d$. It does not perform any computation; it simply passes the features to the first hidden layer.
- **Hidden layers**: one or more intermediate layers that apply non-linear transformations. The neurons in each layer receive the activations from the previous layer as their inputs.
- **Output layer**: produces the final prediction. The number of output neurons and the choice of activation function depend on the task (one neuron with sigmoid for binary classification; K neurons with softmax for K-class classification; one neuron with linear activation for regression).

All connections go from one layer to the next, with no connections within a layer or skipping layers. This is called a *fully connected* (or *dense*) feedforward architecture.

Notation. We use superscripts in parentheses to index layers. For a network with L layers (counting hidden layers and the output layer but not the input):

- $n^{(l)}$ denotes the number of neurons in layer l.
- $W^{(l)} \in \mathbb{R}^{n^{(l)} \times n^{(l-1)}}$ is the weight matrix of layer l.
- $\mathbf{b}^{(l)} \in \mathbb{R}^{n^{(l)}}$ is the bias vector of layer l.
- $\mathbf{z}^{(l)} \in \mathbb{R}^{n^{(l)}}$ is the pre-activation vector of layer l.
- $\mathbf{a}^{(l)} \in \mathbb{R}^{n^{(l)}}$ is the activation vector of layer l, with $\mathbf{a}^{(0)} = \mathbf{x}$.

Figure 4 connects this notation to the structure of a concrete MLP. The input layer contains the original features, while each hidden layer stores a new activation vector computed from the previous layer. The arrows between adjacent layers represent all pairwise connections between neurons, which is why this architecture is called *fully connected*. Each block of arrows corresponds to one weight matrix $W^{(l)}$ and one bias vector $\mathbf{b}^{(l)}$.

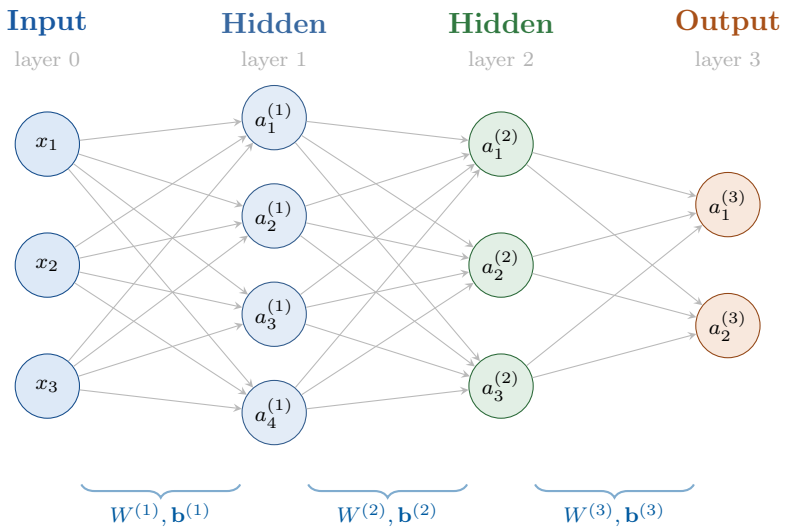

Figure 4: A fully connected MLP with an input layer ($d = 3$), two hidden layers (4 and 3 neurons), and an output layer (2 neurons). Each set of arrows between adjacent layers corresponds to a weight matrix $W^{(l)}$ and a bias vector $\mathbf{b}^{(l)}$.

Counting Parameters

It is important to know how many learnable parameters a given architecture has. Suppose an MLP has 3 input features, one hidden layer with 4 neurons, and an output layer with 2 neurons. The first weight matrix has size 4×3, so it contains 12 weights, and the first bias vector has 4 parameters. The output weight matrix has size $2 \times 4 = 8$ weights, and the output bias vector has 2 parameters. The total number of learnable parameters is therefore

$$12 + 4 + 8 + 2 = 26.$$

In general, the number of parameters between two adjacent layers with n_{in} and n_{out} neurons is $n_{\text{in}} \cdot n_{\text{out}} + n_{\text{out}}$ (weights plus biases).

2.3 Capacity and Depth

Two key architectural choices determine the expressive power of an MLP:

- **Width**: the number of neurons per hidden layer. Wider layers can represent more features simultaneously.
- **Depth**: the number of hidden layers. Deeper networks can learn more hierarchical and abstract representations.

Universal Approximation Theorem

A feedforward network with a single hidden layer containing a finite number of neurons can approximate any continuous function on a compact subset of $\mathbb{R}^d$ to arbitrary precision, given a suitable non-linear activation function [Hornik et al., 1989]. This result guarantees the representational power of MLPs, but says nothing about how to find the right weights or how many neurons are needed. In practice, *deep* networks tend to be more efficient than wide shallow ones for many real-world tasks.

Entry Ticket 2

Consider a network with input dimension $d = 4$, two hidden layers with 8 and 4 neurons respectively, and an output layer with 3 neurons (for a 3-class classification problem).

1. What are the dimensions of each weight matrix $W^{(1)}$, $W^{(2)}$, $W^{(3)}$?
2. How many learnable parameters (weights and biases) does this network have in total?
3. If you double the number of neurons in each hidden layer, how does the total number of parameters change?

3 Activation Functions

3.1 The Role of Non-Linearity

Without activation functions, a network of any depth would reduce to a single linear transformation, since a composition of linear maps is itself linear. Activation functions introduce the non-linearity that allows MLPs to approximate complex functions. The choice of activation function affects the network's expressive power, training speed, and susceptibility to optimization problems.

Why Depth Alone Is Not Enough

Consider a network with two linear layers and no activation function:

$$\mathbf{h} = W^{(1)}\mathbf{x} + \mathbf{b}^{(1)}, \qquad \hat{\mathbf{y}} = W^{(2)}\mathbf{h} + \mathbf{b}^{(2)}.$$

Substituting the first equation into the second gives:

$$\hat{\mathbf{y}} = W^{(2)}W^{(1)}\mathbf{x} + W^{(2)}\mathbf{b}^{(1)} + \mathbf{b}^{(2)}.$$

This is still just a single linear transformation of $\mathbf{x}$ plus a bias term. Therefore, adding more layers without nonlinear activations does not increase the expressive power of the model.

3.2 Common Activation Functions

Table 1 summarizes the most widely used activation functions, their formulas, and derivatives.

Table 1: Common activation functions used in MLPs.

Function	**Formula**	**Derivative**	**Range**
Sigmoid	$\dfrac{1}{1+e^{-z}}$	$\sigma(z)(1-\sigma(z))$	$(0,1)$
Tanh	$\tanh(z)$	$1-\tanh^2(z)$	$(-1,1)$
ReLU	$\max(0,z)$	$\mathbf{1}[z>0]$	$[0,+\infty)$
Leaky ReLU	$\max(\alpha z, z)$, $\alpha \ll 1$	$\mathbf{1}[z>0]+\alpha\,\mathbf{1}[z\leq 0]$	$(-\infty,+\infty)$
Softmax	$\dfrac{e^{z_k}}{\sum_j e^{z_j}}$	Jacobian matrix	$(0,1)^K$, sums to 1

Sigmoid

The sigmoid function maps any real number to the interval $(0,1)$, making it suitable for the output neuron in binary classification. Its derivative is conveniently expressed in terms of its own output:

$$\sigma'(z) = \sigma(z)(1-\sigma(z)).$$

The derivative reaches its maximum value of 0.25 at $z=0$ and approaches zero for large $|z|$, causing the *vanishing gradient problem* in deep networks.

Tanh

The hyperbolic tangent is a scaled and shifted sigmoid: $\tanh(z) = 2\sigma(2z)-1$. Its outputs are centered around zero, which often leads to faster convergence than sigmoid. It still suffers from vanishing gradients in the extremes.

ReLU (Rectified Linear Unit)

ReLU is the most widely used activation in modern deep networks:

$$\mathrm{ReLU}(z) = \max(0,z).$$

It avoids vanishing gradients for positive inputs, is computationally efficient, and leads to sparse activations (many neurons produce exactly zero). A known issue is the *dying ReLU problem*: neurons that always receive negative pre-activations will always output zero and their weights will never be updated.

Softmax (output layer)

For multi-class classification, the softmax function transforms a vector of scores into a probability distribution:

$$\mathrm{softmax}(\mathbf{z})_k = \frac{e^{z_k}}{\sum_{j=1}^{K} e^{z_j}}, \quad k=1,\ldots,K.$$

The output is always non-negative and sums to one, making it interpretable as class probabilities.

Figure 5 provides a visual comparison of the activation functions discussed above. Sigmoid and Tanh compress large positive or negative inputs into saturated regions, where their slopes become very small. ReLU behaves differently: it is exactly zero for negative inputs and linear for positive inputs, which helps preserve gradients when $z > 0$. Leaky ReLU keeps a small negative slope, reducing the risk that a neuron becomes permanently inactive.

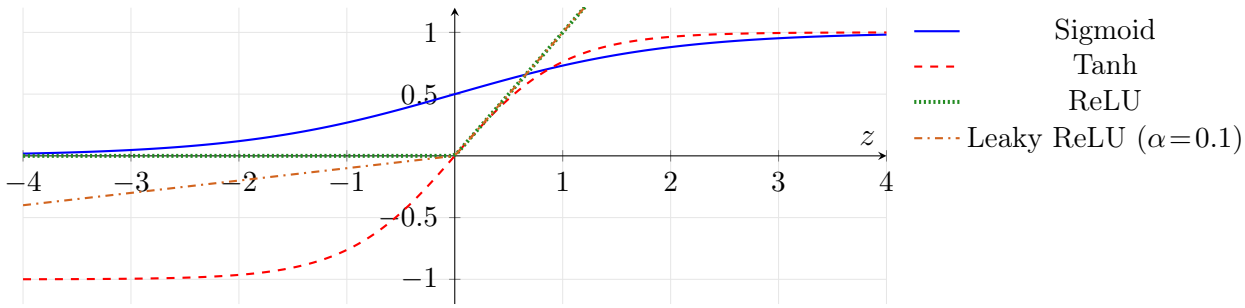

Figure 5: Common activation functions plotted over $z \in [-4, 4]$. ReLU is linear for positive inputs and zero otherwise; Leaky ReLU allows a small negative slope. Sigmoid and Tanh saturate at their extremes, leading to small gradients.

Choosing an Activation Function

In practice, ReLU or one of its variants (Leaky ReLU, ELU, GELU) is the default choice for hidden layers in modern networks, due to its simplicity and resistance to vanishing gradients. For the output layer, the choice depends on the task: sigmoid for binary classification, softmax for multi-class classification, and linear (no activation) for regression.

Entry Ticket 3

A colleague trains an MLP with sigmoid activations on all layers, including several hidden layers. She observes that the network barely improves during training, even after many epochs.

1. What phenomenon might explain this behavior? Relate your answer to the mathematical properties of the sigmoid derivative.
2. How would replacing sigmoid with ReLU help? Are there any new risks introduced by ReLU?
3. In the output layer, she uses ReLU for a binary classification task. Why is this inappropriate? What should she use instead?

4 Forward Pass

The previous sections introduced activation functions as the source of nonlinearity in an MLP, and described how neurons are organized into layers. We now combine these ideas with weight matrices and bias vectors to describe exactly how an input is transformed into a prediction. This computation is called the *forward pass*.

4.1 Layer-by-Layer Computation

Entry Ticket 4

Before reading the worked example, think about the following.

1. If all weights in an MLP are initialized to very large values, what might happen to sigmoid activations in the first few layers?
2. How could this affect the gradients that flow back during training?
3. Based on your answer, why might careful weight initialization matter?

The *forward pass* is the process by which an input $\mathbf{x}$ is propagated through the network to produce a prediction $\hat{\mathbf{y}}$. Each layer applies a linear transformation followed by an element-wise non-linearity:

$$\mathbf{z}^{(l)} = W^{(l)}\mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}, \qquad \mathbf{a}^{(l)} = \sigma\left(\mathbf{z}^{(l)}\right),$$

with $\mathbf{a}^{(0)} = \mathbf{x}$. The final output is $\hat{\mathbf{y}} = \mathbf{a}^{(L)}$.

4.2 Worked Example: Forward Pass in a 2–2–1 Network

Consider a network with two input features, one hidden layer with two neurons, and a single output neuron. All hidden neurons use the sigmoid activation; the output neuron also uses sigmoid (binary classification). The parameters are:

$$W^{(1)} = \begin{bmatrix} 0.15 & 0.20 \\ 0.25 & 0.30 \end{bmatrix}, \quad \mathbf{b}^{(1)} = \begin{bmatrix} 0.35 \\ 0.35 \end{bmatrix}, \quad W^{(2)} = \begin{bmatrix} 0.40 & 0.45 \end{bmatrix}, \quad b^{(2)} = 0.60.$$

Given input $\mathbf{x} = [0.05,\ 0.10]^\top$ and true label $y = 0.01$:

Step 1: Pre-activations of the hidden layer

$$z_1^{(1)} = 0.15(0.05) + 0.20(0.10) + 0.35 = 0.3775,$$
$$z_2^{(1)} = 0.25(0.05) + 0.30(0.10) + 0.35 = 0.3925.$$

Step 2: Activations of the hidden layer

$$a_1^{(1)} = \sigma(0.3775) \approx 0.5932, \qquad a_2^{(1)} = \sigma(0.3925) \approx 0.5969.$$

Step 3: Pre-activation and activation of the output layer

$$z_1^{(2)} = 0.40(0.5932) + 0.45(0.5969) + 0.60 \approx 1.1059,$$
$$\hat{y} = a_1^{(2)} = \sigma(1.1059) \approx 0.7514.$$

The network predicts $\hat{y} \approx 0.751$ for an input with true label 0.01, indicating a large error that training will work to correct.

Figure 6 summarizes the same computation graphically. The numbers inside the neurons correspond to activations, while the annotations above the hidden and output layers show

the pre-activation values z. This distinction is important: the pre-activation is the result of the linear combination, whereas the activation is the value obtained after applying the nonlinear function. During training, both types of quantities are needed because the backward pass uses them to compute gradients.

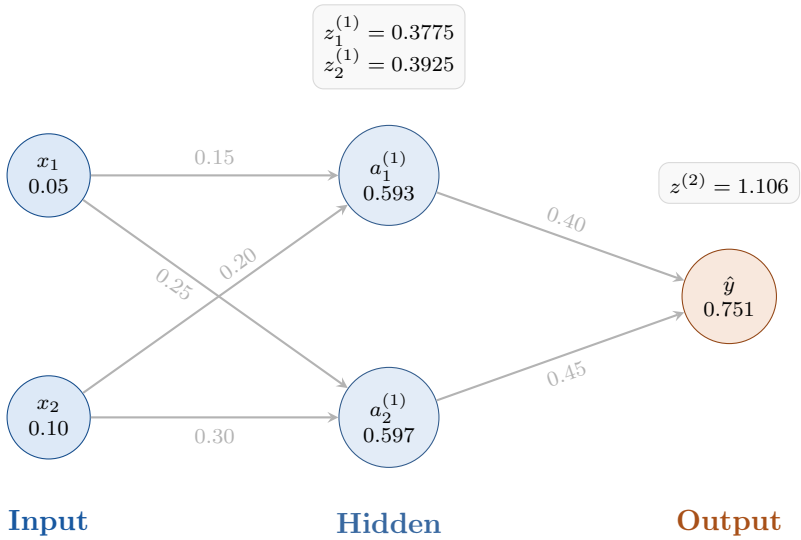

Figure 6: Forward pass in a 2–2–1 MLP. Values inside neurons are activations; rounded boxes show pre-activations z.

5 Loss Functions

5.1 Measuring Prediction Error

To train a network, we need to quantify how far its predictions are from the true labels. A *loss function* $L(\hat{y}, y)$ assigns a scalar value to each prediction-label pair. The training objective is to find network parameters that minimize the average loss over the training set.

Loss Function

A loss function $L : \mathcal{Y} \times \mathcal{Y} \to \mathbb{R}_{\geq 0}$ measures the discrepancy between the predicted output $\hat{y}$ and the true label y. It is differentiable (almost everywhere) with respect to the network parameters, enabling gradient-based optimization.

5.2 Common Loss Functions

Mean Squared Error (regression)

For regression tasks with a single output:

$$L(\hat{y}, y) = \frac{1}{2}(\hat{y} - y)^2.$$

The factor $\frac{1}{2}$ is included purely for algebraic convenience: it cancels with the exponent

when computing the derivative. Over a dataset of m examples:

$$\mathbf{L} = \frac{1}{m} \sum_{i=1}^{m} \frac{1}{2} (\hat{y}^{(i)} - y^{(i)})^2.$$

Binary Cross-Entropy (binary classification)

When $\hat{y} \in (0, 1)$ is interpreted as the probability $P(y = 1|\mathbf{x})$:

$$L(\hat{y}, y) = -\big[y \log \hat{y} + (1 - y) \log(1 - \hat{y})\big].$$

This loss penalizes confident wrong predictions very heavily (as $\hat{y} \to 0$ when $y = 1$, $-\log \hat{y} \to +\infty$).

Categorical Cross-Entropy (multi-class classification)

For K-class problems with a softmax output layer and one-hot label $\mathbf{y}$:

$$L(\hat{\mathbf{y}}, \mathbf{y}) = -\sum_{k=1}^{K} y_k \log \hat{y}_k.$$

Since only the correct class k^* has $y_{k^*} = 1$, this reduces to $-\log \hat{y}_{k^*}$, i.e., the negative log-probability assigned to the true class.

Table 2: Recommended output activations and loss functions for common supervised learning tasks.

Task	**Output activation**	**Loss function**	**Output size**
Regression	Linear (none)	Mean Squared Error	1
Binary classification	Sigmoid	Binary cross-entropy	1
Multi-class classification	Softmax	Categorical cross-entropy	K

Table 2 highlights a practical rule that students should treat as a diagnostic checklist: the output layer and the loss function must be compatible with the learning task. Regression requires unrestricted numerical outputs, so a linear output is appropriate. Binary classification requires a probability for the positive class, typically produced by a sigmoid. Multi-class classification requires a probability distribution over mutually exclusive classes, which is why softmax and categorical cross-entropy are paired.

Mathematical notation vs. implementation. Mathematically, we write cross-entropy losses in terms of probabilities $\hat{y}$ or $\hat{\mathbf{y}}$. In software libraries, however, loss functions may expect either probabilities (the output of a sigmoid or softmax) or *raw scores*, called **logits**. For example, in PyTorch, `BCELoss` expects probabilities (apply sigmoid first),

while `BCEWithLogitsLoss` expects logits and applies the sigmoid internally. Similarly, `CrossEntropyLoss` expects logits and applies softmax internally. This distinction is important and is discussed further in Section 8.

Loss vs. Metric

The **loss function** is what the optimizer minimizes during training; it must be differentiable. A **metric** (e.g., accuracy, F1 score) is what we report to evaluate model quality; it need not be differentiable. For example, we may minimize binary cross-entropy during training but report accuracy as the evaluation metric.

Entry Ticket 5

A student is training an MLP for a 4-class classification problem. The output layer has 4 neurons, followed by sigmoid activations, and the model is trained with mean squared error loss.

1. Why is this combination not appropriate for a standard multi-class problem?
2. What output activation and loss function would you recommend instead?
3. What could go wrong if each output neuron is treated independently with sigmoid in a problem where the classes are mutually exclusive?

6 Training: Gradient Descent and Backpropagation

The forward pass tells us what the network predicts. The loss function tells us how wrong the prediction is. The remaining question is: how should each individual weight and bias be changed to reduce that error? Backpropagation answers this question by computing gradients efficiently via the chain rule.

Common Misconception

Backpropagation is *not* an optimization algorithm by itself. It is an algorithm that computes gradients. A separate optimizer (gradient descent, Adam, etc.) uses those gradients to update the parameters. Many people say "the model was trained with backpropagation," but the more precise statement is "the model was trained using backpropagation to compute gradients and an optimizer to update the weights."

6.1 The Training Objective

Given a training set $\mathcal{D} = \{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^{m}$, the goal of training is to find parameters $\theta = \{W^{(l)}, \mathbf{b}^{(l)}\}_{l=1}^{L}$ that minimize the empirical loss:

$$\hat{\theta} = \arg\min_{\theta} \frac{1}{m} \sum_{i=1}^{m} L\big(\hat{y}^{(i)}, y^{(i)}\big) .$$

Because the loss is a complicated non-convex function of the parameters, there is no closed-form solution. Instead, we use *gradient descent*: iteratively move the parameters in the direction of steepest descent of the loss.

6.2 Gradient Descent

The parameter update rule for gradient descent is:

$$\theta \leftarrow \theta - \eta \nabla_\theta L,$$

where $\eta > 0$ is the *learning rate* and $\nabla_\theta L$ is the gradient of the loss with respect to all parameters. The learning rate controls the step size:

- Too large: the optimizer overshoots minima and may diverge.
- Too small: convergence is slow and the model may get stuck.

Table 3: Main variants of gradient descent according to the number of examples used to estimate each gradient.

Variant	**Gradient computed over**	**Characteristics**
Batch GD	All m training examples	Stable, slow per update
Stochastic GD (SGD)	One random example at a time	Noisy, fast, good regularization
Mini-batch GD	A random subset of B examples	Balance of stability and speed

Variants of gradient descent. Table 3 compares three ways of estimating the gradient during training. Batch gradient descent uses the entire training set before each update, producing stable but expensive updates. Stochastic gradient descent updates the parameters after a single example, which makes each update cheap but noisy. Mini-batch gradient descent is the standard compromise in neural networks: it uses enough examples to obtain a useful gradient estimate while still allowing frequent and computationally efficient updates.

In practice, *mini-batch gradient descent* is the standard. A typical mini-batch size B is between 32 and 512. Each pass through the entire training set is called an *epoch*.

6.3 Backpropagation: Computing Gradients Efficiently

To apply gradient descent, we need $\nabla_\theta L$, which includes the gradient with respect to every weight and bias in the network. For a network with millions of parameters, computing these gradients naively would be intractable. **Backpropagation** is an efficient algorithm that computes all gradients in a single backward pass through the network, using the chain rule of calculus.

Intuition before the equations. Before looking at the formulas, it is useful to think of backpropagation as a bookkeeping procedure. During the forward pass, the network records all intermediate computations. During the backward pass, it asks: if the final loss changed slightly, how much responsibility should each weight and bias receive for that change? This question is answered layer by layer, starting from the output and working backward to the input.

Backpropagation

Backpropagation is an algorithm for computing the gradient of the loss function with respect to all parameters of a neural network. It applies the chain rule layer by layer, propagating error signals from the output back to the input, and computing the gradient for each parameter along the way. Its time complexity is $\mathcal{O}(|\theta|)$: proportional to the number of parameters.

6.4 The Chain Rule and Error Signals

The chain rule is the mathematical foundation of backpropagation. For a composed function $L = L(a(z(w)))$:

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial a} \cdot \frac{\partial a}{\partial z} \cdot \frac{\partial z}{\partial w}.$$

Each factor has a concrete interpretation in the MLP:

- $\dfrac{\partial z}{\partial w} = a^{(l-1)}$ (the activation feeding into the weight).
- $\dfrac{\partial a}{\partial z} = \sigma'(z)$ (the derivative of the activation function).
- $\dfrac{\partial L}{\partial a}$ (the error signal arriving from the layer above).

The **error signal** (or *delta*) for layer l is defined as:

$$\boldsymbol{\delta}^{(l)} = \frac{\partial L}{\partial \mathbf{z}^{(l)}},$$

and it satisfies the backward recursion:

$$\boldsymbol{\delta}^{(l)} = \left[\left(W^{(l+1)}\right)^{\top} \boldsymbol{\delta}^{(l+1)}\right] \odot \sigma'\left(\mathbf{z}^{(l)}\right),$$

where $\odot$ denotes element-wise multiplication. This is the key equation of backpropagation: the error signal at layer l is obtained by (i) passing the error from layer $l+1$ backward through the transpose weight matrix, then (ii) element-wise multiplying by the local derivative of the activation function.

Check Your Understanding

In the equation for $\boldsymbol{\delta}^{(l)}$, identify which term depends on the layer above and which term depends only on the current layer. Why does this structure make backpropagation a *recursive* algorithm? What information must be stored during the forward pass to enable this computation?

6.5 The Training Loop: Forward, Backward, and Update

The training of an MLP is not just backpropagation. A complete training iteration has three phases. First, the network performs a *forward pass*, computing predictions and storing intermediate quantities. Second, backpropagation performs a *backward pass*, computing error signals and gradients. Third, an optimizer uses those gradients to update

the parameters. Thus, backpropagation is the gradient-computation step inside the larger training loop.

Forward Values, Backward Sensitivities

A useful way to distinguish the three parts of training is the following:

- the **forward pass** computes values: pre-activations, activations, predictions, and loss;
- the **backward pass** computes sensitivities: how much the loss would change if each intermediate quantity or parameter changed slightly;
- the **optimizer** uses these sensitivities, expressed as gradients, to update the parameters.

In short: the forward pass tells us *what the network predicts*, the backward pass tells us *which quantities are responsible for the error*, and the optimizer decides *how to change the parameters*.

Phase 1: Forward Pass

For $l = 1, 2, \ldots, L$:

$$\mathbf{z}^{(l)} = W^{(l)}\mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}, \qquad \mathbf{a}^{(l)} = \sigma\big(\mathbf{z}^{(l)}\big)\,.$$

Compute and store all $\mathbf{z}^{(l)}$ and $\mathbf{a}^{(l)}$.

Phase 2: Backward Pass

Compute the output error signal:

$$\boldsymbol{\delta}^{(L)} = \nabla_{\mathbf{a}^{(L)}} L \odot \sigma'\big(\mathbf{z}^{(L)}\big)\,.$$

For $l = L-1, L-2, \ldots, 1$, propagate backward:

$$\boldsymbol{\delta}^{(l)} = \Big[\big(W^{(l+1)}\big)^\top \boldsymbol{\delta}^{(l+1)}\Big] \odot \sigma'\big(\mathbf{z}^{(l)}\big)\,.$$

Compute gradients for each layer:

$$\frac{\partial L}{\partial W^{(l)}} = \boldsymbol{\delta}^{(l)}\,\big(\mathbf{a}^{(l-1)}\big)^\top, \qquad \frac{\partial L}{\partial \mathbf{b}^{(l)}} = \boldsymbol{\delta}^{(l)}.$$

Phase 3: Parameter Update

$$W^{(l)} \leftarrow W^{(l)} - \eta\,\frac{\partial L}{\partial W^{(l)}}, \qquad \mathbf{b}^{(l)} \leftarrow \mathbf{b}^{(l)} - \eta\,\frac{\partial L}{\partial \mathbf{b}^{(l)}}.$$

Figure 7 organizes the full training procedure as a repeated loop. First, the network performs a forward pass to compute predictions and the loss. Then, backpropagation computes the error signals and gradients in the reverse direction. Finally, the optimizer updates the parameters. This cycle is repeated for many mini-batches and epochs, gradually reducing the loss when the learning rate and model configuration are appropriate.

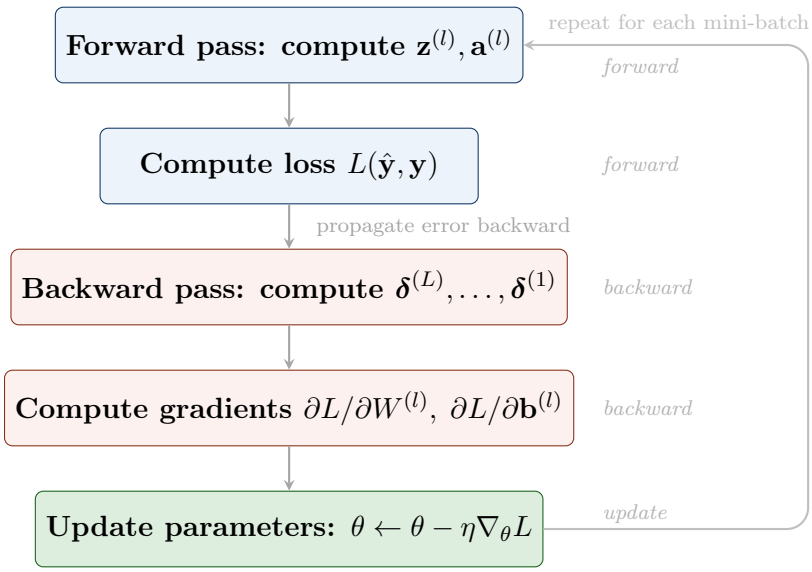

Figure 7: The training loop for an MLP. Each iteration (one mini-batch) consists of a forward pass to compute predictions and loss, a backward pass to compute gradients via backpropagation, and a parameter update step. This loop repeats for multiple epochs until convergence.

6.6 Worked Example: Backward Pass

We now continue the 2–2–1 example from Section 4.2, where $\hat{y} = 0.7514$, $y = 0.01$, and the loss function is MSE. The forward-pass example computed the numerical values of all activations. We now reuse those values to compute how much each parameter contributed to the final error. The key quantity is the error signal δ, which measures how sensitive the loss is to a neuron's pre-activation z. Once δ is known for a neuron, the gradient of each incoming weight is obtained by multiplying that error signal by the activation that entered through that weight.

Output error signal

We begin at the output neuron because this is where the prediction is directly compared with the true label. However, the quantity used by backpropagation is not only the prediction error $\hat{y} - y$. The error signal $\delta^{(2)}$ measures the sensitivity of the loss with respect to the pre-activation $z^{(2)}$. Therefore, it must account for both the loss derivative and the sigmoid derivative:

$$\delta^{(2)} = \frac{\partial L}{\partial z^{(2)}} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z^{(2)}}.$$

For MSE loss with sigmoid output, this gives:

$$\delta^{(2)} = (\hat{y} - y) \cdot \hat{y}(1 - \hat{y}) = (0.7414)(0.1868) \approx 0.1385.$$

Gradient for output weights

Once the error signals have been computed, each weight gradient follows the same pattern: multiply the activation that entered the weight by the error signal of the neuron that

received it.

$$\frac{\partial L}{\partial W^{(2)}} = \delta^{(2)} \left(\mathbf{a}^{(1)}\right)^\top = 0.1385 \cdot [0.5932,\ 0.5969] \approx [0.0822,\ 0.0826].$$

Hidden layer error signals

The hidden neurons do not produce the final prediction directly, so their error signals cannot be computed by comparing their activations with the true label. Instead, each hidden neuron receives a portion of the output error, weighted by how strongly it contributed to the output neuron. This propagated error is then multiplied by the local derivative of the activation function, because a hidden neuron can only influence the loss through its pre-activation and activation. In this example, the output error $\delta^{(2)}$ is first passed backward through $W^{(2)}$, and then adjusted by the sigmoid derivative of each hidden neuron.

$$\boldsymbol{\delta}^{(1)} = \left(W^{(2)}\right)^\top \delta^{(2)} \odot \mathbf{a}^{(1)}\left(1 - \mathbf{a}^{(1)}\right) = \begin{bmatrix} 0.0554 \\ 0.0623 \end{bmatrix} \odot \begin{bmatrix} 0.2410 \\ 0.2405 \end{bmatrix} \approx \begin{bmatrix} 0.0134 \\ 0.0150 \end{bmatrix}.$$

The Backpropagation Identity

The gradient with respect to any weight $w_{ij}^{(l)}$ factorizes as:

$$\frac{\partial L}{\partial w_{ij}^{(l)}} = a_i^{(l-1)} \cdot \delta_j^{(l)},$$

where $a_i^{(l-1)}$ is the activation feeding into the weight, and $\delta_j^{(l)}$ is the error signal at the neuron the weight feeds into. This compact formula makes the implementation of backpropagation straightforward as a sequence of matrix operations.

Gradient for hidden layer weights

$$\frac{\partial L}{\partial W^{(1)}} = \boldsymbol{\delta}^{(1)}\mathbf{x}^\top = \begin{bmatrix} 0.0134 \\ 0.0150 \end{bmatrix} [0.05,\ 0.10] = \begin{bmatrix} 0.00067 & 0.00134 \\ 0.00075 & 0.00150 \end{bmatrix}.$$

After one update step with $\eta = 0.5$, the loss decreases from approximately 0.2748 to 0.2710, confirming that the gradients point in the right direction.

Entry Ticket 6

Consider an MLP being trained with gradient descent and backpropagation.

1. In the backward pass, the gradient for $W^{(1)}$ depends on $\boldsymbol{\delta}^{(1)}$, which in turn depends on $\boldsymbol{\delta}^{(2)}$. Why is this a problem for very deep networks using sigmoid activations?
2. Explain in your own words why storing the intermediate activations $\mathbf{a}^{(l)}$ during the forward pass is necessary for the backward pass.
3. If the learning rate η is set too large, what can go wrong during training? Describe what you would observe in the loss curve.

Figure 8 summarizes the backward pass for the same network used in the forward-pass example. The structure of the graph is the same, but the meaning of the numbers has changed: the neuron values now represent error signals, and the edge labels represent gradients with respect to the weights.

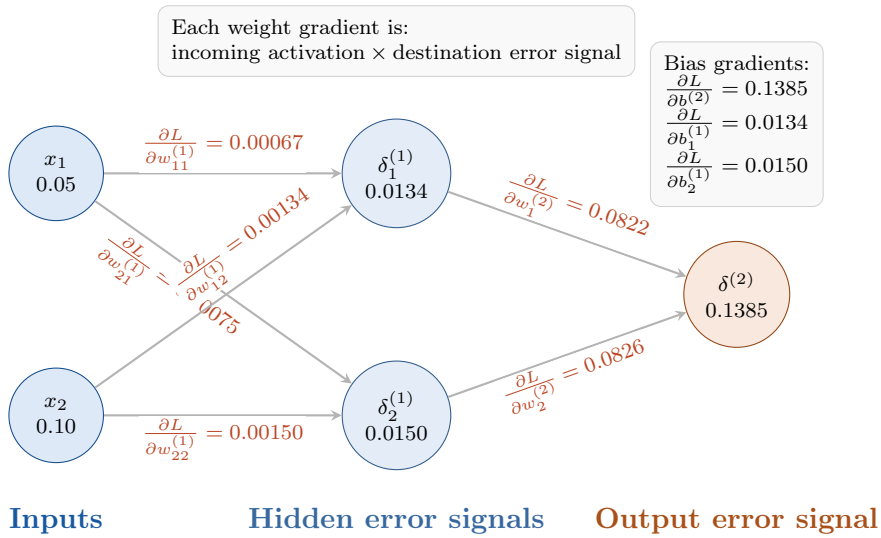

Figure 8: Backward pass in the same 2–2–1 MLP used in the forward-pass example. Values inside hidden and output neurons are error signals δ, while edge labels show gradients of the loss with respect to the corresponding weights.

7 Optimization in Practice

7.1 Weight Initialization

The initial values of the weights matter significantly. If all weights are initialized to zero, all neurons in a layer will compute the same gradient and remain identical throughout training, a phenomenon called *symmetry breaking failure*. Weights should be initialized to small random values. Two principled initialization schemes are:

- **Xavier / Glorot initialization** (recommended for Tanh/Sigmoid):

$$w \sim \mathcal{U}\left[-\sqrt{\frac{6}{n_{\text{in}}+n_{\text{out}}}},\ \sqrt{\frac{6}{n_{\text{in}}+n_{\text{out}}}}\right].$$

- **He initialization** (recommended for ReLU):

$$w \sim \mathcal{N}\left(0,\ \frac{2}{n_{\text{in}}}\right).$$

Both schemes set the variance of the weights so that the signal neither amplifies nor vanishes as it passes through layers.

7.2 Advanced Optimizers

Plain gradient descent is rarely used in practice. Modern optimizers adapt the effective learning rate for each parameter, leading to faster and more stable convergence.

Optimizer	Key idea	When to prefer
SGD + Momentum	Accumulates a velocity vector to smooth updates	Simple tasks, fine-tuned LR
RMSProp	Divides LR by exponential moving average of squared gradients	Recurrent networks
Adam	Combines momentum and per-parameter adaptive LR	Strong default for most tasks

Adam (Adaptive Moment Estimation) maintains a running estimate of both the first and second moments of the gradient:

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t, \qquad v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2,$$

$$\theta_t = \theta_{t-1} - \frac{\eta}{\sqrt{\hat{v}_t} + \epsilon} \hat{m}_t,$$

where $\hat{m}_t$ and $\hat{v}_t$ are bias-corrected moment estimates. Default values $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$ work well across many tasks. Adam is often a strong default for initial experimentation, although carefully tuned SGD with momentum can sometimes achieve better generalization in specific settings.

7.3 Regularization

MLPs are prone to overfitting, especially with large capacity and small datasets. Common regularization techniques include:

L2 Regularization (Weight Decay)

Add a penalty on the squared magnitude of weights to the loss:

$$L_{\text{reg}} = L + \frac{\lambda}{2} \sum_{l,i,j} \left(w_{ij}^{(l)} \right)^2 .$$

This discourages large weights and reduces model complexity.

Dropout

During training, each neuron's output is randomly set to zero with probability p (the *dropout rate*). In modern frameworks such as PyTorch, the remaining activations are scaled by a factor of $1/(1-p)$ during training; this convention is called *inverted dropout*. At test time, dropout is disabled and all neurons contribute normally. Conceptually, dropout prevents the network from relying too heavily on any single neuron, forcing it to learn redundant and more robust representations.

Early Stopping

Monitor the validation loss during training. Stop training when the validation loss begins to increase, even if the training loss continues to decrease. This prevents the network from memorizing the training set.

Data Augmentation

When the problem domain allows it, artificial training examples can be created by applying label-preserving transformations to existing examples. For images, this may include small rotations, crops, or changes in brightness. For tabular data, data augmentation is less straightforward and must be applied with care.

Figure 9 illustrates the logic behind early stopping. The training loss keeps decreasing because the model continues to fit the training data more closely. The validation loss, however, eventually starts to increase, indicating that the model is beginning to specialize too much to the training set. The best stopping point is therefore not necessarily the final epoch, but the epoch with the lowest validation loss.

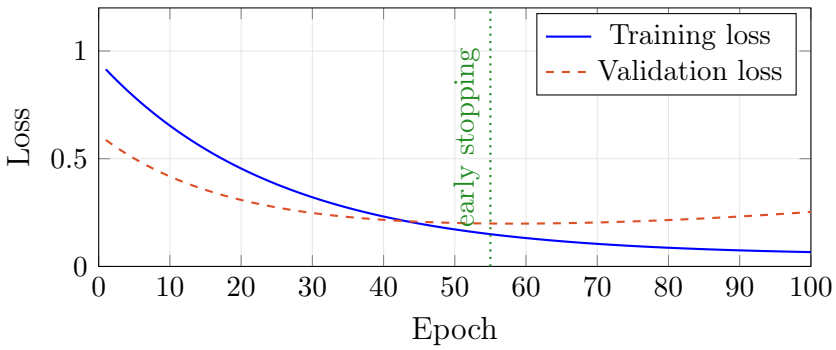

Figure 9: Illustration of early stopping. The training loss decreases monotonically, while the validation loss begins to rise after a certain number of epochs due to overfitting. Training should stop at the epoch where the validation loss is minimized.

The Bias–Variance Trade-off in MLPs

The fundamental tension in MLP design mirrors the bias-variance trade-off:

- A *small* network (few layers, few neurons per layer) has high bias: it underfits because it cannot represent the complexity of the true function.
- A *large* network has low bias but high variance: it may memorize the training data and fail to generalize.

Regularization techniques (dropout, weight decay, early stopping) reduce variance without significantly increasing bias. Careful architecture choice balances the two.

Entry Ticket 7

A practitioner trains a large MLP on a dataset with 500 training examples. After 200 epochs, the training accuracy is 99% but the validation accuracy is 65%.

1. What is happening? Name the phenomenon and explain it in terms of bias and variance.
2. Suggest three different interventions she could apply, and explain the mechanism by which each one would help.
3. She adds a dropout layer with $p = 0.9$ (dropping 90% of neurons). Is this a reasonable choice? What would you recommend instead?

8 Implementation

8.1 Implementation in Scikit-learn

Scikit-learn provides the `MLPClassifier` and `MLPRegressor` classes for quick experimentation with MLPs. Listing 1 illustrates a complete basic workflow for classification: splitting the data, scaling the input features, defining the MLP architecture, training the model, and reporting predictive performance. The use of `make_pipeline` is important because it ensures that preprocessing and model fitting are treated as a single reproducible procedure.

```python
from sklearn.neural_network import MLPClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, classification_report

# Split data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Build pipeline: scale inputs, then MLP
mlp = make_pipeline(
    StandardScaler(),
    MLPClassifier(
        hidden_layer_sizes=(128, 64),  # two hidden layers
        activation='relu',             # hidden layer activation
        solver='adam',                 # optimizer
        learning_rate_init=1e-3,       # initial learning rate
        max_iter=300,                  # maximum epochs
        early_stopping=True,           # stop if val loss stops improving
        validation_fraction=0.1,       # fraction used for early stopping
        random_state=42
    )
)

mlp.fit(X_train, y_train)
y_pred = mlp.predict(X_test)

print(f"Accuracy: {accuracy_score(y_test, y_pred):.3f}")
print(classification_report(y_test, y_pred))
```

Listing 1: MLP classifier in scikit-learn with preprocessing and evaluation.

In this example, the test set is held out until the final evaluation. When `early_stopping` is set to `True`, scikit-learn internally splits a fraction of `X_train`, controlled by the value of `validation_fraction`, to monitor validation performance during training. The test set is used only for the final performance report and should never be used for model selection,

early stopping, or hyperparameter tuning.

Feature Scaling is Essential

MLPs are sensitive to the scale of input features. Inputs with very different magnitudes cause the optimizer to behave poorly: gradients from large-scale features dominate updates, while small-scale features are largely ignored. Always apply `StandardScaler` (or similar normalization) before feeding data to an MLP. This is not optional.

8.2 Implementation in PyTorch

For research and production deep learning, **PyTorch** and **TensorFlow/Keras** are the standard frameworks. They provide automatic differentiation (autograd), GPU support, and a vast ecosystem of tools. Listing 2 presents the same basic MLP idea in PyTorch, but at a lower level of abstraction than scikit-learn. The code explicitly defines the neural network as a subclass of `nn.Module`, constructs the sequence of linear layers and ReLU activations, chooses a loss function and optimizer, and then executes the training loop manually. This makes the relationship between forward pass, loss computation, backpropagation, and parameter update more visible.

```python
import torch
import torch.nn as nn
import torch.optim as optim

# --- Define the MLP architecture ---
class MLP(nn.Module):
    def __init__(self, input_dim, hidden_dims, output_dim):
        super().__init__()
        layers = []
        prev_dim = input_dim
        for h in hidden_dims:
            layers += [nn.Linear(prev_dim, h), nn.ReLU()]
            prev_dim = h
        layers.append(nn.Linear(prev_dim, output_dim))
        self.net = nn.Sequential(*layers)

    def forward(self, x):
        return self.net(x)

# --- Instantiate model, loss, optimizer ---
model     = MLP(input_dim=10, hidden_dims=[128, 64], output_dim=3)
criterion = nn.CrossEntropyLoss()   # expects logits, not probs.
optimizer = optim.Adam(model.parameters(), lr=1e-3)

# --- Training loop ---
for epoch in range(100):
    model.train()
    optimizer.zero_grad()           # clear previous gradients
    outputs = model(X_train_t)      # forward pass (outputs are
```

```
        logits)
    loss = criterion(outputs, y_train_t)
    loss.backward()                    # backward pass (backpropagation)
    optimizer.step()                   # update parameters

    if (epoch + 1) % 10 == 0:
        print(f"Epoch {epoch+1:3d} | Loss: {loss.item():.4f}")
```

Listing 2: MLP for classification implemented in PyTorch.

The call `loss.backward()` triggers automatic differentiation: PyTorch traces all operations in the forward pass and computes gradients for every parameter that requires a gradient. This is the framework-level implementation of backpropagation.

Why There Is No Softmax Layer in the PyTorch Code

In PyTorch, `nn.CrossEntropyLoss()` expects raw class scores (*logits*) as input. It internally applies `log_softmax` followed by the negative log-likelihood loss. Therefore, when using `CrossEntropyLoss`, we should **not** add an explicit `nn.Softmax` layer at the end of the model. Adding both would produce incorrect gradient magnitudes and is one of the most common beginner mistakes. If you want to obtain class probabilities at inference time, apply `torch.softmax(outputs, dim=1)` after the forward pass, outside the loss computation.

What This Minimal PyTorch Example Omits

The code above is intentionally minimal for clarity. A production-quality training loop would typically include mini-batches with `DataLoader`, a separate validation set evaluated inside a `torch.no_grad()` block to avoid computing unnecessary gradients, model checkpointing, a learning rate scheduler, and early stopping. Refer to the PyTorch documentation for complete training recipes.

9 Common Pitfalls and Debugging

9.1 First Sanity Checks

Before tuning architecture settings or optimizer hyperparameters, always verify the basics:

- Check whether the input features contain missing values or extreme outliers, and apply the appropriate preprocessing.
- Verify that labels are encoded in the format expected by the loss function (e.g., integer class indices for `CrossEntropyLoss`, not one-hot vectors).
- Test whether the model can overfit a very small subset of the training data (e.g., 10 examples). If it cannot, something is wrong with the architecture or the loss.
- Confirm that the train, validation, and test splits were created without data leakage.
- Check that the loss decreases meaningfully during the first few epochs.

9.2 Checklist for Training MLPs

Training a neural network involves many interacting design choices. When a network does not perform as expected, the checklist presented in Table 4 helps identify the issue. Table 4 should be read as a first diagnostic map rather than as a complete troubleshooting manual. Each symptom points to a likely class of problems, but several causes may occur simultaneously. In practice, debugging an MLP usually starts with simple checks, such as verifying the loss/output combination and feature scaling, before moving to more expensive changes such as modifying the architecture or collecting more data.

Table 4: Common symptoms, likely causes, and possible fixes when training MLPs.

Symptom	**Likely cause**	**Fix**
Loss does not decrease	LR too small, bad init, wrong loss	Increase LR, check loss/output match
Loss diverges (NaN)	LR too large, no normalization	Reduce LR, add `StandardScaler`
High train loss, high val loss	Model too small (underfitting)	Add layers/neurons
Low train loss, high val loss	Overfitting	Dropout, L2, early stopping, more data
Slow convergence	Bad optimizer or LR schedule	Switch to Adam, use LR scheduling
All outputs same class	Dead neurons, initialization	Use He init with ReLU
Predictions look random	Labels encoded incorrectly or wrong loss	Check label format and loss function
Val loss $\ll$ test loss	Distribution shift or test set overused	Check splits, use test set only once

9.3 The Vanishing Gradient Problem

As discussed in Section 3.2, sigmoid and tanh activations saturate for large $|z|$, producing near-zero derivatives. When gradients are multiplied through many layers (via the chain rule), they may shrink exponentially:

$$\frac{\partial L}{\partial \mathbf{z}^{(1)}} = \left[\prod_{l=2}^{L} \left(W^{(l)}\right)^{\top} \operatorname{diag}\left(\sigma'(\mathbf{z}^{(l-1)})\right)\right] \frac{\partial L}{\partial \mathbf{z}^{(L)}}.$$

If each $\sigma'(z) < 0.25$ and $\|W^{(l)}\| \approx 1$, the product of these small factors causes the gradient at early layers (those close to the input) to be orders of magnitude smaller than at the output layer. Layers close to the input may effectively receive almost no training signal.

Solutions.

- Use ReLU activations (gradient is 1 for positive inputs, never saturates).
- Use **batch normalization**: normalize activations within each mini-batch to zero mean and unit variance, preventing saturation.
- Use **residual connections** (skip connections), which allow gradients to flow directly across layers without passing through all activation functions.

Practical Guidelines for MLP Design

- **Activation**: use ReLU (or Leaky ReLU/GELU) for hidden layers.
- **Output**: sigmoid (binary), softmax (multi-class), or linear (regression).
- **Initialization**: He for ReLU, Xavier for Tanh/Sigmoid.
- **Optimizer**: Adam with learning rate 10^{-3} as a starting point.
- **Scaling**: always normalize inputs to zero mean and unit variance.
- **Regularization**: early stopping is the cheapest option; add dropout if overfitting persists.
- **Architecture**: start shallow and wide, then go deeper only if necessary.
- **Loss/output match**: always verify that the output activation and loss function correspond to your task.

10 Connection to Deep Learning

10.1 MLPs as the Foundation of Deep Learning

The MLP, as studied in these notes, is the simplest instance of a deep learning model. The conceptual framework it establishes (layered computation, non-linear activations, gradient-based learning via backpropagation) generalizes directly to all modern deep learning architectures:

Table 5: How common deep learning architectures extend the basic MLP idea.

Architecture	**Key extension over MLP**	**Application domain**
Convolutional Neural Network (CNN)	Local connectivity + weight sharing	Images, video
Recurrent Neural Network (RNN)	Connections through time	Sequences, time series
Transformer	Self-attention mechanism	Text, images, audio
Graph Neural Network (GNN)	Aggregation over graph structure	Molecular data, social networks

Table 5 shows that many modern architectures can be understood as specialized versions of the same basic idea: compose parameterized transformations and train them with gradients. CNNs modify the connectivity pattern to exploit spatial structure, RNNs

introduce recurrence for sequential data, Transformers use attention to model relationships between tokens, and GNNs adapt the same learning principle to graph-structured data.

Every architecture in Table 5 still relies on:

- forward passes through parameterized layers with non-linear activations;
- a differentiable loss function;
- backpropagation to compute gradients;
- a gradient-based optimizer (typically Adam) to update parameters.

The differences lie in the *structure* of the weight matrices and the connections between layers, not in the fundamental learning mechanism.

10.2 What Changes at Scale

Modern deep learning models may have billions of parameters and are trained on massive datasets. While the mathematics remains the same, several additional engineering concerns arise:

- **Hardware**: training requires GPUs or TPUs, which parallelize matrix operations.
- **Memory**: storing all intermediate activations for backpropagation is expensive; techniques like gradient checkpointing trade computation for memory.
- **Stability**: very deep networks require careful initialization, normalization layers, and learning rate scheduling to converge reliably.
- **Generalization**: with sufficient capacity, deep networks can memorize any training set; regularization and data augmentation become critical.

From MLP to Deep Learning

The journey from understanding MLPs to deploying state-of-the-art deep learning models is largely a matter of (i) architectural specialization (convolutions, attention), (ii) engineering (distributed training, hardware acceleration), and (iii) empirical methodology (hyperparameter tuning, ablation studies). The mathematical core studied in these notes (forward pass, loss, backpropagation, gradient descent) remains unchanged.

Entry Ticket 8

Consider a Convolutional Neural Network (CNN) used for image classification.

1. A CNN still uses backpropagation and gradient descent. What does this imply about the operations performed in each layer of a CNN?
2. An MLP applied to a $224 \times 224 \times 3$ image (the standard ImageNet resolution) would have $224 \times 224 \times 3 = 150{,}528$ input features. What architectural problem does this create, and how do CNNs address it?
3. Name one practical advantage of having studied MLPs thoroughly before moving on to more complex architectures.

11 Final Remarks

11.1 Summary of Key Concepts

The following points summarize the essential content of these notes:

- An MLP is a layered, fully connected feedforward neural network. Each layer applies a linear transformation followed by a non-linear activation function.
- Linear models learn linear boundaries in the given feature space; MLPs **learn their own feature representations** through hidden layers, enabling them to discover nonlinear structure automatically.
- The **forward pass** propagates inputs through the network layer by layer to produce a prediction. All intermediate values $(\mathbf{z}^{(l)}, \mathbf{a}^{(l)})$ are stored for the backward pass.
- The **loss function** quantifies the error between the prediction and the true label. The choice of loss and output activation must match the task.
- **Backpropagation** efficiently computes the gradient of the loss with respect to all parameters using the chain rule. It is an algorithm for computing gradients, not an optimizer. The key operation is the propagation of error signals $\boldsymbol{\delta}^{(l)}$ from output to input.
- **Gradient descent** (typically mini-batch, using Adam) iteratively updates parameters to minimize the loss.
- **Overfitting** is the primary risk for large MLPs; it is controlled through dropout, weight decay, early stopping, and data augmentation.
- The MLP is the conceptual foundation for all modern deep learning architectures.

11.2 Recommended Next Steps

These notes prepare the student for the following topics in the deep learning module:

$$\text{MLP} \rightarrow \text{CNNs} \rightarrow \text{Sequence Models (RNN/LSTM)} \rightarrow \text{Attention \& Transformers} \rightarrow \text{Generative Models}$$

Key Takeaways

- MLPs learn hierarchical, non-linear representations through stacked layers.
- The universal approximation theorem guarantees expressive power, but finding the right parameters requires gradient-based optimization.
- Backpropagation makes training tractable by computing all gradients in a single backward pass using the chain rule.
- Practical success depends on careful choices of activation function, initialization, optimizer, and regularization.
- All modern deep learning architectures are extensions of the MLP principle: composition of parameterized, differentiable transformations trained by gradient descent.

11 References

Kurt Hornik, Maxwell Stinchcombe, and Halbert White. Multilayer feedforward networks are universal approximators. *Neural Networks*, 2(5):359–366, 1989. doi: 10.1016/0893-6080(89)90020-8.

David E. Rumelhart, Geoffrey E. Hinton, and Ronald J. Williams. Learning representations by back-propagating errors. *Nature*, 323(6088):533–536, 1986. doi: 10.1038/323533a0.